

Database Management Systems - Quiz Solutions (Set 2)

1. Difference between Join and Cartesian Product

Cartesian Product ($\times$):

- **Definition:** It combines **every** tuple of one relation with **every** tuple of another relation.
- **Result Size:** If relation R has m tuples and S has n tuples, the result has $m \times n$ tuples.
- **Significance:** It is a blind combination without any matching criteria.

Join ($\bowtie$):

- **Definition:** It combines tuples from two relations based on a specific **condition** (usually matching values in common columns).
- **Result Size:** It is a subset of the Cartesian product, containing only meaningful combinations.
- **Significance:** It reconstructs data related across tables.

2. Derive Intersection in terms of Union and Set Difference

The Intersection operation ($R \cap S$) returns tuples that are present in both relations. It is not a fundamental operator and can be derived as follows:

$$R \cap S = R - (R - S)$$

Explanation:

1. $(R - S)$ gives tuples that are in R but **not** in S .
2. Subtracting that result from R removes the unique tuples, leaving only those that **are** also in S .

Alternative derivation:

$$R \cap S = (R \cup S) - ((R - S) \cup (S - R))$$

3. SQL Query: Pattern Matching

Question: Find all students in relation `student(name, department, city)` whose names start with "S" and end with "a".

```
SELECT * FROM student
WHERE name LIKE 'S%a';
```

- `%` is the wildcard for zero or more characters.

4. SQL Query: NULL Check

Question: Retrieve names of employees from `employee(FName, ..., SuperSSN, ...)` who do not have supervisors.

```
SELECT FName, L_name
FROM employee
WHERE SuperSSN IS NULL;
```

- Note: You must use `IS NULL`, not `= NULL`.

5. Relational Algebra Query

Question: Find names of employees in 'CSE' department earning > 50,000 from `employee(EID, Name, Department, Salary)`.

$$\pi_{Name}(\sigma_{Department='CSE' \wedge Salary > 50000}(Employee))$$

- σ (Sigma): Selects rows matching the condition.
- π (Pi): Projects (selects) the `Name` column.

6. Basic SQL Constraint Types

SQL supports the following basic integrity constraints to ensure data accuracy and reliability:

1. **NOT NULL:** Ensures a column cannot have a NULL value.
2. **UNIQUE:** Ensures all values in a column are different.
3. **PRIMARY KEY:** A combination of `NOT NULL` and `UNIQUE`. Uniquely identifies each row.
4. **FOREIGN KEY:** Ensures referential integrity; a value must match a value in another table's primary key.
5. **CHECK:** Ensures that the values in a column satisfy a specific condition.
6. **DEFAULT:** Sets a default value for a column if no value is specified.

7. Find Functional Dependencies (FDs)

Given Instance: | A | B | C | D | |---|---|---|---| | a1 | b2 | c2 | d1 | | a1 | b2 | c3 | d2 | | a1 | b1 | c3 | d2 | | a2 | b3 | c2 | d3 |

Analysis of Single-Attribute FDs:

1. **Check $A \rightarrow \dots$**
 - $a1$ maps to $b2$ and $b1$. (Violates $A \rightarrow B$)
 - $a1$ maps to $c2$ and $c3$. (Violates $A \rightarrow C$)
 - $a1$ maps to $d1$ and $d2$. (Violates $A \rightarrow D$)

2. Check $B \rightarrow \dots$

- $B \rightarrow A$:
 - $b2 \rightarrow a1$ (Row 1, 2)
 - $b1 \rightarrow a1$ (Row 3)
 - $b3 \rightarrow a2$ (Row 4)
 - **Holds.**
- $B \rightarrow C: b2 \rightarrow \{c2, c3\}$. (Violates)
- $B \rightarrow D: b2 \rightarrow \{d1, d2\}$. (Violates)

3. Check $C \rightarrow \dots$

- $c2 \rightarrow \{a1, a2\}$. (Violates $C \rightarrow A$)
- $c2 \rightarrow \{b2, b3\}$. (Violates $C \rightarrow B$)
- $c2 \rightarrow \{d1, d3\}$. (Violates $C \rightarrow D$)

4. Check $D \rightarrow \dots$

- $D \rightarrow A$:
 - $d1 \rightarrow a1$
 - $d2 \rightarrow a1$ (Row 2, 3)
 - $d3 \rightarrow a2$
 - **Holds.**
- $D \rightarrow C$:
 - $d1 \rightarrow c2$
 - $d2 \rightarrow c3$ (Row 2, 3)
 - $d3 \rightarrow c2$
 - **Holds.**
- $D \rightarrow B: d2 \rightarrow \{b2, b1\}$. (Violates)

Valid FDs found:

- $B \rightarrow A$
- $D \rightarrow A$
- $D \rightarrow C$
- (By transitivity, since $D \rightarrow C$ and $D \rightarrow A$, then $D \rightarrow AC$ also holds).

Test: University Library Database

Schema:

- Book (Book_ID, title, author, publisher, year)
- Member (Member_ID, name, address)

- Borrow (Member_ID, Book_ID, borrow_date, return_date)

1. Relational Algebra Queries

(i) Retrieve the names of members who have borrowed the book titled "Database Systems."

$$\pi_{name}(Member \bowtie (Borrow \bowtie \sigma_{title='DatabaseSystems'}(Book)))$$

(ii) Find the titles of books borrowed by the member named Tom.

$$\pi_{title}(Book \bowtie (Borrow \bowtie \sigma_{name='Tom'}(Member)))$$

(iii) Retrieve the names of members who have borrowed more than three distinct books.

Using Aggregate Functions:

$$T_1 \leftarrow \text{Member_ID} \bowtie \text{COUNT(Book_ID)}(Borrow)$$

$$T_2 \leftarrow \sigma_{count > 3}(T_1)$$

$$Result \leftarrow \pi_{name}(T_2 \bowtie Member)$$

(iv) Retrieve the names of members who have borrowed at least one book from every author present in the book relation. *Logic:* This requires the **Division** operation ($\div$). We need to find members whose set of "borrowed authors" covers the set of "all authors".

1. Get (Member, Author) pairs:

$$R_1 \leftarrow \pi_{Member_ID, author}(Borrow \bowtie Book)$$

2. Get All Authors:

$$R_2 \leftarrow \pi_{author}(Book)$$

3. Perform Division (Members who borrowed from ALL authors):

$$R_3 \leftarrow R_1 \div R_2$$

4. Get Names:

$$Result \leftarrow \pi_{name}(R_3 \bowtie Member)$$

2.a. Extensions to Join: Outer Joins (Relational Algebra)

Standard joins (Inner Joins) discard tuples that do not match the join condition. **Outer Joins** are extensions designed to keep all tuples from one or both relations, padding with `NULL` where no match is found.

i) Left Outer Join ($R \setminus \text{leftouterjoin} S$):

- **Definition:** Keeps every tuple in the **Left** relation (R). If a tuple in R has no match in S , the attributes of S are filled with `NULL`.
- **Example:** Listing all Employees, and their Department Name if assigned (otherwise `NULL`).

ii) Right Outer Join ($R \setminus \text{rightouterjoin} S$):

- **Definition:** Keeps every tuple in the **Right** relation (S). If a tuple in S has no match in R , the attributes of R are filled with `NULL`.
- **Example:** Listing all Departments, and their Manager's name if one exists (otherwise `NULL`).

iii) Full Outer Join ($R \setminus \text{fullouterjoin} S$):

- **Definition:** Keeps **all** tuples from both R and S . It is the union of Left and Right Outer Joins. Unmatched tuples from either side are padded with `NULL`.

Example Tables:

- **Emp (Name, DeptID):** { (Alice, 101), (Bob, NULL) }
- **Dept (DeptID, DName):** { (101, HR), (102, IT) }
- **Left Join (Emp $\setminus \text{leftouterjoin}$ Dept):**
 - (Alice, 101, HR)
 - (Bob, NULL, NULL) -> Bob preserved.
- **Right Join (Emp $\setminus \text{rightouterjoin}$ Dept):**
 - (Alice, 101, HR)
 - (NULL, 102, IT) -> IT preserved.

2.b. Purpose and Usage of Six SQL Clauses

The standard SQL query structure follows the order: `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY`.

1. SELECT:

- **Purpose:** Specifies the attributes (columns) to be retrieved in the result set.
- **Usage:** `SELECT Name, Salary` or `SELECT *` (for all columns).

2. FROM:

- **Purpose:** Specifies the table(s) or views from which data is to be retrieved. Joins are often defined here.
- **Usage:** `FROM Employee` or `FROM Employee E JOIN Department D ON ...`.

3. WHERE:

- **Purpose:** Filters **individual rows** based on a specified condition *before* any grouping is done.
- **Usage:** WHERE Salary > 50000 AND DeptID = 10 .

4. GROUP BY:

- **Purpose:** Groups rows that have the same values in specified columns into summary rows (used with aggregate functions like COUNT, MAX, AVG).
- **Usage:** GROUP BY DeptID (Calculates aggregates per department).

5. HAVING:

- **Purpose:** Filters **groups** based on a condition *after* grouping is done. (Unlike WHERE, it can filter on aggregates).
- **Usage:** HAVING COUNT(*) > 5 (Only show departments with more than 5 employees).

6. ORDER BY:

- **Purpose:** Sorts the final result set in Ascending (ASC) or Descending (DESC) order.
- **Usage:** ORDER BY Salary DESC .

Example: Query: Find departments with an average salary greater than 60,000, sorted by department ID.

SELECT DeptID, AVG(Salary)	-- 1. SELECT columns
FROM Employee	-- 2. FROM table
WHERE Salary > 0	-- 3. WHERE filter (ignore invalid salary)
GROUP BY DeptID	-- 4. GROUP BY department
HAVING AVG(Salary) > 60000	-- 5. HAVING filter on aggregate
ORDER BY DeptID ASC;	-- 6. ORDER BY result

3.a. Equivalence of Functional Dependency Sets

Problem: Determine if $F \equiv G$ for relation $R(A, B, C, D)$.

- $F = \{A \rightarrow B, B \rightarrow C, AB \rightarrow D\}$
- $G = \{A \rightarrow B, B \rightarrow C, A \rightarrow C, A \rightarrow D\}$

Solution: Two sets of FDs are equivalent if $F \subseteq G^+$ (every FD in F is implied by G) AND $G \subseteq F^+$ (every FD in G is implied by F).

Step 1: Check if $F \subseteq G^+$

- $A \rightarrow B$: Exists in G . (Holds)
- $B \rightarrow C$: Exists in G . (Holds)
- $AB \rightarrow D$: Check closure $(AB)^+$ using G .
 - Start: $\{A, B\}$
 - Using $A \rightarrow D$ (from G): Add D .
 - Result: $\{A, B, D\}$. Since D is in closure, $AB \rightarrow D$ holds.

Step 2: Check if $G \subseteq F^+$

- $A \rightarrow B$: Exists in F . (Holds)
- $B \rightarrow C$: Exists in F . (Holds)
- $A \rightarrow C$: Check closure $(A)^+$ using F .
 - Start: $\{A\}$
 - Using $A \rightarrow B$ (from F): Add B .
 - Using $B \rightarrow C$ (from F): Add C .
 - Result: $\{A, B, C\}$. Since C is in closure, $A \rightarrow C$ holds.
- $A \rightarrow D$: Check closure $(A)^+$ using F .
 - Start: $\{A\}$
 - Using $A \rightarrow B$: Add B . Result $\{A, B\}$.
 - Using $AB \rightarrow D$ (from F): Since we have A and B , Add D .
 - Result: $\{A, B, D\}$. Since D is in closure, $A \rightarrow D$ holds.

Conclusion: Since both conditions are satisfied, **F and G are Equivalent Sets.**

3.b. Minimal Cover Calculation

Problem: Find the minimal cover for relation $R(A, B, C, D, E, F)$ with FDs:

$$F = \{A \rightarrow BC, C \rightarrow D, AB \rightarrow C, D \rightarrow A, A \rightarrow B\}$$

Solution:

Step 1: Decompose FDs into Canonical Form (Single RHS) Break $A \rightarrow BC$ into $A \rightarrow B$ and $A \rightarrow C$.

- $F' = \{A \rightarrow B, A \rightarrow C, C \rightarrow D, AB \rightarrow C, D \rightarrow A, A \rightarrow B\}$
- Remove exact duplicate $A \rightarrow B$.
- $F' = \{A \rightarrow B, A \rightarrow C, C \rightarrow D, AB \rightarrow C, D \rightarrow A\}$

Step 2: Remove Extraneous Attributes (LHS Simplification) Check $AB \rightarrow C$.

- Can we simplify AB to A or B ?
- Does $A \rightarrow C$ hold in the rest of the set? Yes, $A \rightarrow C$ is explicitly present.
- Therefore, $AB \rightarrow C$ is redundant because $A \rightarrow C$ is stronger. We can remove $AB \rightarrow C$ entirely (or view it as reducing to $A \rightarrow C$ which is a duplicate).
- $F'' = \{A \rightarrow B, A \rightarrow C, C \rightarrow D, D \rightarrow A\}$

Step 3: Remove Redundant FDs Check each FD to see if it can be derived from the others.

- **Check $A \rightarrow B$:** Can we derive B from A using $\{A \rightarrow C, C \rightarrow D, D \rightarrow A\}$?
 - $(A)^+ = \{A, C, D\}$. B is not found. Keep $A \rightarrow B$.
- **Check $A \rightarrow C$:** Can we derive C from A using $\{A \rightarrow B, C \rightarrow D, D \rightarrow A\}$?

- $(A)^+ = \{A, B\}$. C is not found. Keep $A \rightarrow C$.
- **Check $C \rightarrow D$:** Can we derive D from C using others?
 - $(C)^+ = \{C\}$. No. Keep.
- **Check $D \rightarrow A$:** Can we derive A from D using others?
 - $(D)^+ = \{D\}$. No. Keep.

Final Minimal Cover: $\{A \rightarrow B, A \rightarrow C, C \rightarrow D, D \rightarrow A\}$

Note: You may recombine them for conciseness: $\{A \rightarrow BC, C \rightarrow D, D \rightarrow A\}$.

4. SQL Queries on Company Database

Assumed Schema:

- EMPLOYEE (Fname, Lname, Ssn, ..., Dno, Super_ssn)
- DEPARTMENT (Dname, Dnumber, ...)
- PROJECT (Pname, Pnumber, Plocation, Dnum)
- WORKS_ON (Essn, Pno, Hours)
- DEPENDENT (Essn, Dependent_name, ...)

a. Retrieve the names of all employees in Department 5 who work more than 10 hours per week on the ProductX project.

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE E
JOIN WORKS_ON W ON E.Ssn = W.Essn
JOIN PROJECT P ON W.Pno = P.Pnumber
WHERE E.Dno = 5
      AND P.Pname = 'ProductX'
      AND W.Hours > 10;
```

b. List the names of all employees who have a dependent with the same first name as themselves.

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE E
JOIN DEPENDENT D ON E.Ssn = D.Essn
WHERE E.Fname = D.Dependent_name;
```

c. Find the names of all employees who are directly supervised by 'Franklin Wong'.

Concept: This requires a **Self-Join**. We join the Employee table to itself, treating one instance as the "Employee" (E) and the other as the "Supervisor" (S).

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE E
```



```

JOIN EMPLOYEE S ON E.Super_ssn = S.Ssn
WHERE S.Fname = 'Franklin'
      AND S.Lname = 'Wong';

```

d. Find the names of all employees who work on every project controlled by department number 5. *Concept:* This is the SQL equivalent of the **Division** operator. We are looking for employees such that *there does not exist* a project in Dept 5 that the employee *does not work on*.

Approach 1: Using NOT EXISTS (Double Negation)

```

SELECT Fname, Lname
FROM EMPLOYEE E
WHERE NOT EXISTS (
    SELECT *
    FROM PROJECT P
    WHERE P.Dnum = 5
    AND NOT EXISTS (
        SELECT *
        FROM WORKS_ON W
        WHERE W.Essn = E.Ssn
        AND W.Pno = P.Pnumber
    )
);

```

Approach 2: Using EXCEPT (Set Difference)

```

SELECT Fname, Lname
FROM EMPLOYEE E
WHERE NOT EXISTS (
    (SELECT Pnumber FROM PROJECT WHERE Dnum = 5)
    EXCEPT
    (SELECT Pno FROM WORKS_ON WHERE Essn = E.Ssn)
);

```

4. SQL Queries on Company Database

****Assumed Schema:****

```

* `EMPLOYEE` (Fname, Lname, Ssn, ..., Dno, Super_ssn)`
* `DEPARTMENT` (Dname, Dnumber, ...)`
* `PROJECT` (Pname, Pnumber, Plocation, Dnum)`
* `WORKS ON` (Essn, Pno, Hours)`
* `DEPENDENT` (Essn, Dependent_name, ...)`

```

****a.** Retrieve the names of all employees in Department 5 who work more than 10 hc

```

```sql
SELECT E.Fname, E.Lname
FROM EMPLOYEE E
JOIN WORKS_ON W ON E.Ssn = W.Essn
JOIN PROJECT P ON W.Pno = P.Pnumber
WHERE E.Dno = 5
 AND P.Pname = 'ProductX'

```

AND W.Hours > 10;

**b. List the names of all employees who have a dependent with the same first name as themselves.**

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE E
JOIN DEPENDENT D ON E.Ssn = D.Essn
WHERE E.Fname = D.Dependent_name;
```

**c. Find the names of all employees who are directly supervised by 'Franklin Wong'.**

*Concept:* This requires a **Self-Join**. We join the Employee table to itself, treating one instance as the "Employee" (*E*) and the other as the "Supervisor" (*S*).

```
SELECT E.Fname, E.Lname
FROM EMPLOYEE E
JOIN EMPLOYEE S ON E.Super_ssn = S.Ssn
WHERE S.Fname = 'Franklin'
 AND S.Lname = 'Wong';
```

**d. Find the names of all employees who work on every project controlled by department number 5.** *Concept:* This is the SQL equivalent of the **Division** operator. We are looking for employees such that *there does not exist* a project in Dept 5 that the employee *does not work on*.

**Approach 1: Using NOT EXISTS (Double Negation)**

```
SELECT Fname, Lname
FROM EMPLOYEE E
WHERE NOT EXISTS (
 SELECT *
 FROM PROJECT P
 WHERE P.Dnum = 5
 AND NOT EXISTS (
 SELECT *
 FROM WORKS ON W
 WHERE W.Essn = E.Ssn
 AND W.Pno = P.Pnumber
)
);
```

**Approach 2: Using EXCEPT (Set Difference)**

```
SELECT Fname, Lname
FROM EMPLOYEE E
WHERE NOT EXISTS (
```

```

(SELECT Pnumber FROM PROJECT WHERE Dnum = 5)
EXCEPT
(SELECT Pno FROM WORKS_ON WHERE Essn = E.Ssn)
);

```

## 5. Analysis of Relation Instance

**Given Instance:** | a | b | c | d | |---|---|---|---| | 1 | x | p | 100 | | 1 | x | q | 100 | | 1 | x | q | 100 | | 2 | y | q | 200 | | 2 | y | q | 200 | | 3 | z | r | 300 | | 3 | z | r | 400 |

**a) Identify all non-trivial functional dependencies.** We analyze the data for patterns where a determinant set uniquely maps to a dependent attribute.

1. **Check  $A \rightarrow B$ :**
  - $1 \rightarrow x, 2 \rightarrow y, 3 \rightarrow z$ . (Holds)
2. **Check  $B \rightarrow A$ :**
  - $x \rightarrow 1, y \rightarrow 2, z \rightarrow 3$ . (Holds)
3. **Check  $D \rightarrow A$ :**
  - $100 \rightarrow 1, 200 \rightarrow 2, 300 \rightarrow 3, 400 \rightarrow 3$ . (Holds)
4. **Check  $D \rightarrow B$ :**
  - Implied by  $D \rightarrow A$  and  $A \rightarrow B$ . (Holds)
5. **Check  $A \rightarrow C$  (and others involving C):**
  - $1 \rightarrow \{p, q\}$ . (Violated)
  - $D \rightarrow C: 100 \rightarrow \{p, q\}$ . (Violated)
6. **Check  $A \rightarrow D$ :**
  - $3 \rightarrow \{300, 400\}$ . (Violated)

**Non-Trivial FDs:**

- $A \rightarrow B$
- $B \rightarrow A$
- $D \rightarrow A$  (and by transitivity  $D \rightarrow B$ )

**b) Determine the candidate keys for R.** A candidate key must determine all attributes  $\{a, b, c, d\}$ .

- **Attribute C:** Does not appear on the RHS of any valid FD. Therefore, **C must be part of any key.**
- **Attribute D:** Does not appear on the RHS of any valid FD (unlike A and B, which are determined by D). Therefore, **D must be part of any key.**
- **Test  $\{C, D\}$ :**
  - $D \rightarrow A$  (Given)
  - $A \rightarrow B$  (Given)

- Therefore,  $\{C, D\} \rightarrow \{C, D, A, B\}$ .
- This is a Super Key.
- **Minimality:**
  - $\{C\}$  alone determines nothing.
  - $\{D\}$  alone determines  $A, B$  but not  $C$ .
- **Conclusion:** The only Candidate Key is  **$\{C, D\}$** .

**c) State the highest normal form satisfied by R.**

- **Candidate Key:**  $\{C, D\}$
- **Prime Attributes:**  $C, D$
- **Non-Prime Attributes:**  $A, B$
- **Check 2NF:** A relation is in 2NF if it has **no partial dependencies** (non-prime attribute depending on a proper subset of the key).
  - Here, we have the FD  $D \rightarrow A$ .
  - $D$  is a proper subset of the key  $\{C, D\}$ .